

1 实验一：并行遗传算法求解旅行商问题 (TSP)

1.1 算法背景与分布式并行思路

本实验针对经典的旅行商问题 (TSP)，采用分布式并行遗传算法 (Island Model GA) 进行优化。相比于传统的串行遗传算法，并行方案通过多个进程（岛屿）独立进化，并定期进行优良个体迁移，能够有效维持种群多样性，跳出局部最优解。

并行参数设计：

- **迁移间隔 (Migration Interval)**: 每隔一定代数（如 2000 代）进行一次数据交换。
- **迁移数量 (Migration Count)**: 每次迁移 2 个优良个体。
- **迁移拓扑 (Migration Topology)**: 实现了 island (All-to-All) 和 chain (Ring) 两种模式。
- **迁移策略 (Migration Strategy)**: 选择当前种群中最优个体进行迁移 (best)。

1.2 核心代码实现与注释

并行化改进主要体现在 main 函数中的迁移触发和独立的 migrate_if_needed 函数。

```
1 // 主循环中的迁移调用
2 for (GenNum = 0; GenNum < maxGen; GenNum++) {
3     for (int i = 0; i < xColony; i++)
4         local_improve_one(i); // 执行局部搜索优化个体路径
5     select1_local(); // 比较新老个体，保留距离更短的解
6     migrate_if_needed(rank, size); // 到达迁移代数时，在进程间交换优良个体
7 }
8
9 // 迁移函数的并行实现
10 static void migrate_if_needed(int rank, int size) {
11     if (GenNum % migration_interval != 0) return;
12
13     // 选出最优个体索引并打包
14     int send_idx[N_COLONY];
15     pick_k_indices(send_idx, migration_count, 1 /* pick_best */);
16
17     int send_buf[N_COLONY * CITY];
18     for (int m = 0; m < migration_count; m++)
19         memcpy(&send_buf[m * xCity], colony[send_idx[m]], xCity * sizeof(
20             int));
21
22     if (topo == TOPO_CHAIN_RING) {
23         // 环形拓扑：使用 MPI_Sendrecv 实现相邻进程数据双向交换
```

```

23     MPI_Sendrecv(send_buf, migration_count * xCity, MPI_INT, next, 100,
24                 recv_buf, migration_count * xCity, MPI_INT, prev, 100,
25                 MPI_COMM_WORLD, &st);
26 } else {
27     // 全连接拓扑: 使用 MPI_Allgather 汇总所有进程的优良个体
28     MPI_Allgather(send_buf, migration_count * xCity, MPI_INT,
29                 all_buf, migration_count * xCity, MPI_INT,
30                 MPI_COMM_WORLD);
31 }
32 // 注入接收到的外部基因, 替换本地最差个体
33 replace_k_individuals(recv_buf, migration_count, 1 /* replace_worst */)
34 ;
35 }

```

Listing 1: 并行 TSP 核心逻辑与注释 (mpitsp.c)

1.3 实验结果与统计分析

实验采用 pcb442.tsp 实例。通过对 res/results_raw.txt 中的串行 (Serial) 与并行 (MPI, P=4) 各 30 次实验数据进行统计, 得到如下结果。

1.3.1 统计计算公式

1. 样本均值 ($\bar{x}$):

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

2. 样本标准差 (s):

$$s = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1}}$$

3. 独立样本 t 检验统计量 (t): 针对两组样本量相同 ($n_1 = n_2 = 30$) 且方差接近的情况, 使用合并方差 s_p^2 :

$$s_p^2 = \frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}$$

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{s_p^2 \left(\frac{1}{n_1} + \frac{1}{n_2} \right)}}$$

1.3.2 实验数据与计算结果

根据实验设计, 我们在 pcb442.tsp 实例上对串行与并程序分别运行了 30 次 ($n_1 = n_2 = 30$)。详细原始数据记录于 res/results_raw.txt。

平均提升分析:

- 均值差值 ($\bar{x}_{serial} - \bar{x}_{mpi}$) = 235.333
- 相对解质量提升 $\approx 0.456\%$

统计项	串行 (Serial)	MPI (岛模型)
样本数 n	30	30
平均值 $\bar{x}$	51587.133	51351.800
样本方差 s^2	20837.568	17816.924
最好成绩 (Min)	51241.0	51103.0

表 1: TSP 实验统计结果对比表

1.3.3 Welch t 检验显著性分析

考虑到两组样本方差不完全相等的情况，本实验采用更为稳健的 **Welch 双样本 t 检验**（双侧检验）。

检验设置：

- 原假设 $H_0: \mu_{serial} = \mu_{mpi}$ （两组总体均值相同）
- 备择假设 $H_1: \mu_{serial} \neq \mu_{mpi}$ （两组总体均值不同）

计算过程：代入实验统计量到 Welch t 检验公式：

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}} = 6.556078$$

$$df \approx \frac{\left(\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}\right)^2}{\frac{(s_1^2/n_1)^2}{n_1-1} + \frac{(s_2^2/n_2)^2}{n_2-1}} = 57.648$$

得到计算结果：

- t -统计量：6.556078
- p -value： 1.6646×10^{-8}

1.3.4 结论分析

在显著性水平 $\alpha = 0.05$ （甚至 $\alpha = 0.01$ ）下，由于 $p\text{-value} \ll \alpha$ ，我们 **** 拒绝原假设 H_0 ****。这在统计学上极有力地证明了 **** 并行遗传算法（MPI 岛模型）得到的解质量显著优于串行算法 ****。

并行化带来的解质量提升主要归功于：

- **多路径搜索：**并行环境下的多个岛屿从不同的随机空间开始探索，显著降低了陷入局部最优的风险。
- **迁移机制的协同效应：**通过 Chain 拓扑定期交换最优个体，使得各个进程能够共享进化路径中的优良基因，从而在有限的迭代次数内更接近全局最优解。

2 实验二：并行 K-means 聚类算法

2.1 数据集说明与实际意义

本次实验使用 Iris (鸢尾花) 数据集。

- **数据特征**：包含 150 个样本，4 个维度（花瓣/花萼的长宽）。
- **实际意义**：鸢尾花数据集是聚类分析的标准 benchmark。在此数据集上运行 K-means 的实际意义在于通过无监督学习，仅凭借植物形态学尺寸，自动将 150 朵花归类为 3 个物种。这对生物分类、自动识别等领域具有研究价值。

2.2 并行化设计思想

在 K-means 的并行化中，我们面临两种选择：1. **任务并行**（每个进程负责一个类中心）：不可取，因为 $K = 3$ 无法利用多核优势。2. **数据并行**（每个进程负责部分样本点的距离计算）：**本实验采用的方案**。

具体实现步骤：

- **分配阶段**：每个进程仅处理 $1/P$ 规模的数据点。
- **局部统计**：每个进程计算本地样本点到 K 个中心的最近距离，并累加到本地的 `sum_local` 和 `cnt_local` 数组中。
- **全局同步**：利用 `MPI_Allreduce` 函数。将所有进程的局部和进行累加汇总，并同步到所有进程。其操作类似于先求和再广播。

2.3 并行核心代码实现

```
1 // 1. 各进程计算本地归属并统计部分和
2 for (int i = begin; i < end; i++) {
3     int bestk = find_nearest_centroid(X[i], C);
4     cnt_local[bestk]++;
5     for (int j = 0; j < DIM; j++) sum_local[bestk][j] += X[i][j];
6 }
7
8 // 2. 使用 MPI_Allreduce 实现全局规约更新
9 // 将各进程算得的坐标总和 sum_local 累加到 sum_global
10 MPI_Allreduce(sum_local, sum_global, K*DIM, MPI_DOUBLE, MPI_SUM,
11               MPI_COMM_WORLD);
12 // 将各进程算得的计数 cnt_local 累加到 cnt_global
13 MPI_Allreduce(cnt_local, cnt_global, K, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
14
15 // 3. 所有进程同步更新中心点
16 for (int k = 0; k < K; k++) {
```

```

16     for (int j = 0; j < DIM; j++) C[k][j] = sum_global[k][j] / cnt_global[k]
17 }

```

Listing 2: 并行 K-means 数据规约逻辑 (mpikmeans.c)

2.4 实验结果展示

通过在本地环境运行串行与并程序 ($P = 4$), 得到的结果如下:

2.4.1 串行 K-means 运行结果

```

1 Serial k-means on iris: n=149, k=3, dim=4
2 C0 (count=39): 6.8538 3.0769 5.7154 2.0538
3 C1 (count=60): 5.8983 2.7483 4.4067 1.4417
4 C2 (count=50): 5.0060 3.4280 1.4620 0.2460
5 SSE=76.467679

```

2.4.2 并行 MPI K-means 运行结果

```

1 MPI k-means on iris: n=149, k=3, dim=4, P=4
2 C0 (count=39): 6.8538 3.0769 5.7154 2.0538
3 C1 (count=60): 5.8983 2.7483 4.4067 1.4417
4 C2 (count=50): 5.0060 3.4280 1.4620 0.2460
5 SSE=76.467679
6 Parallel strategy: data-parallel (each rank handles ~n/P points, Allreduce
  sums/counts)

```

2.5 结果分析

从运行结果可以看出: 1. **数值一致性**: 并行版本与串行版本得到的各个簇的中心点坐标、样本计数以及最终的误差平方和 (SSE) 完全一致。这证明了基于 `MPI_Allreduce` 的全局规约思路在逻辑上是严谨且正确的。

2. **并行优势**: 并行程序通过 n/P 的数据划分, 使得每个进程计算距离的开销降低到原来的四分之一。对于像 Iris 这样的小规模数据集, 加速效果虽然在毫秒级不明显, 但对于百万量级的大数据, 这种“数据并行”的架构将展现出显著的性能提升。

3 实验分析与结论

1. **解质量提升**: 并行 GA 的“岛屿模型”由于维持了多个独立的进化搜索路径, 并通过迁移增强了信息交换, 有效避免了串行 GA 易陷入局部最优的问题。T 检验结果明确证明了并行化带来的质量飞跃。

2. **并行效率**: 并行 K-means 通过数据均衡划分和高效的规约操作 (Allreduce), 消除了计算瓶颈, 适合大规模聚类任务。通过本次实验, 我们深入理解了如何利用 MPI 的点对点通信 (Sendrecv) 和集体通信 (Allreduce) 来改造传统算法, 实现高性能计算。